Week-1

1# Prepare a report on SageMath tool and its different functionalities.

Introduction to SageMath-SageMath (short for *System for Algebra and Geometry Experimentation*) is an open-source mathematical software system designed to support research, education, and advanced computation in mathematics. It integrates many existing open-source mathematics packages into a unified Python-based interface. SageMath provides tools for symbolic computation, numerical analysis, algebra, geometry, combinatorics, cryptography, statistics, and more.Unlike proprietary systems such as MATLAB, Mathematica, or Maple, SageMath is free and open-source, making it widely accessible for academic and professional use.

Core Features of SageMath

1 Symbolic Computation

SageMath supports symbolic mathematics through integration with libraries such as SymPy and Maxima. Users can:

- Perform symbolic differentiation and integration
- Simplify algebraic expressions
- Expand and factor polynomials
- Solve algebraic equations symbolically
- Work with symbolic limits and series expansions

Example capabilities:

- Compute derivatives and integrals of symbolic functions
- Solve systems of equations
- Manipulate expressions involving variables

2 Algebra

SageMath is particularly strong in algebraic structures and abstract algebra. It supports:

- Polynomial rings
- Groups, rings, and fields
- Linear algebra operations
- Matrix computations
- Finite fields
- Algebraic number theory

Users can define custom algebraic structures and perform operations such as computing eigenvalues, determinants, inverses, and solving linear systems.

3 Calculus and Analysis

SageMath provides tools for both symbolic and numerical calculus, including:

- Limits
- Differentiation and integration
- Multivariable calculus
- Differential equations
- Taylor and power series expansions

It can solve ordinary differential equations symbolically or numerically and visualize solutions graphically.

4 Numerical Computation

For numerical work, SageMath integrates powerful numerical libraries. Features include:

- Floating-point and arbitrary precision arithmetic
- Numerical integration
- Root finding
- Optimization
- Numerical linear algebra

It supports high-precision calculations, which are essential in scientific computing.

5 Graph Theory

SageMath includes extensive tools for graph theory:

- Creation and manipulation of graphs
- Directed and undirected graphs
- Graph traversal algorithms
- Shortest path algorithms
- Graph coloring
- Connectivity analysis

6 Combinatorics

SageMath has advanced combinatorics capabilities, including:

- Permutations and combinations
- Partitions
- Combinatorial structures
- Generating functions
- Counting techniques

It is widely used for research in algebraic combinatorics.

7 Geometry and Topology

SageMath supports:

- Euclidean and non-Euclidean geometry
- Polyhedra
- Algebraic geometry
- Computational topology

Users can define geometric objects and perform transformations and visualizations.

8 Number Theory

SageMath is highly regarded for its number theory functionality:

- Modular arithmetic
- Prime number testing
- Integer factorization

- Cryptographic algorithms
- Elliptic curves
- Diophantine equations

It is frequently used in cryptographic research and computational number theory.

9 Statistics and Data Analysis

SageMath includes statistical tools such as:

- Descriptive statistics
- Probability distributions
- Hypothesis testing
- Random variable simulation

It can also interface with other statistical packages.

10 Visualization and Plotting

SageMath offers strong 2D and 3D plotting capabilities:

- Function plots
- Parametric plots
- Implicit plots
- Surface plots
- Interactive visualizations

Plots can be exported in various formats for publication.

Programming Interface

SageMath uses Python as its primary programming language. Users can:

- Write scripts
- Create functions and modules
- Automate computations
- Combine symbolic and numerical workflows

Because it is Python-based, it can integrate with other Python libraries such as NumPy and Matplotlib.

User Interface

SageMath provides multiple ways to interact:

- Command-line interface
- Jupyter Notebook interface
- Web-based Sage Notebook
- Integration with other IDEs

The Jupyter interface is especially popular in educational settings.

Applications of SageMath

- Academic research
- University teaching

- Engineering mathematics
- Cryptography
- Data science
- Scientific computing

Advantages of SageMath

- Free and open-source
- Integrates many powerful mathematical libraries
- Python-based and highly extensible
- Strong support for algebra and number theory
- Active research and educational community

Conclusion-SageMath is a comprehensive and powerful open-source mathematics software system. Its integration of numerous mathematical libraries under a Python-based framework makes it versatile and accessible. With capabilities ranging from symbolic algebra and calculus to number theory, graph theory, statistics, and visualization, SageMath serves as a valuable tool for students, educators, researchers, and professionals in mathematics and related fields.

Overall, SageMath stands as a cost-effective and highly capable alternative to commercial mathematical software systems.

2# Write a Python program to detect whether given integer is perfect or not ?

```
def perfect(n):
    sum = 0
    for i in range(1, n):
        if n % i == 0:
            sum += i

    if sum == n and n != 0:
        return "Perfect number"
    else:
        return "Not a perfect number"

num = int(input("Enter number: "))
print(perfect(num))
```

```
Enter number: 6
Perfect number
```

3# Write a Python program to detect whether given integer is Armstrong Integer or not?

```
num = int(input("Enter an integer: "))
original_num = num
num_digits = len(str(num))
sum_of_powers = 0
while num > 0:
    digit = num % 10
    sum_of_powers += digit ** num_digits
    num = num // 10
if sum_of_powers == original_num:
    print(original_num, "is an Armstrong number.")
else:
    print(original_num, "is not an Armstrong number.")
```

```
Enter an integer: 12
12 is not an Armstrong number.
```

Week-2

1# Write a program to detect whether a number is prime or not.

```
n = int(input("Enter a number: "))
if n <= 1:
    print("Not a prime number")
else:
    for i in range(2, n):
        if n % i == 0:
            print("Not a prime number")
            break
    else:
        print("Prime number")
```

```
============== RESTART:
Enter a number: 5
Prime number
```

2# Write a function to compute GCD of two integers.

```
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

a = int(input("Enter number a: "))
b = int(input("Enter number b: "))

print("GCD is:", gcd(a, b))
```

```
============== RESTAR
Enter number a: 4
Enter number b: 8
GCD is: 4
```

3# Write a program to detect whether two numbers are relatively prime or not?

```
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

a = int(input("Enter number a: "))
b = int(input("Enter number b: "))

g = gcd(a, b)
#print("GCD is:", g)

if g == 1:
    print("The numbers are relatively prime.")
else:
    print("The numbers are not relatively prime.")
```

```
Enter number a: 12
Enter number b: 8
The numbers are not relatively prime.
>>>
```

4# Write a function to display all'n' narcissistic number. For example, 153 is a 3 narcissistic number because 13 + 53 + 33 = 153, and 1634 is 4 narcissistic number because 14 + 64 + 34 + 44 = 1634.

```
def display_narcissistic(n):
    start = 10 ** (n - 1)
    end = 10 ** n

    for num in range(start, end):
        temp = num
        total = 0
        while temp > 0:
            digit = temp % 10
            total += digit ** n
            temp //= 10
        if total == num:
            print(num, end=", ")
n = int(input("Enter value of n: "))

display_narcissistic(n)
```

Week-3

1# Write a function to check whether a number is in the form of 2k or not.

```
def form():
    n = int(input("Enter a number: "))
    k = 0
    temp = n
    while temp > 1 and temp % 2 == 0:
        temp = temp // 2
        k += 1
    if temp == 1:
        print(f"{n} is in the form of 2^k (for k = {k})")
    else:
        print(f"{n} is NOT in the form of 2^k")
```

```
    Enter a number: 12
    12 is NOT in the form of 2^k
>>>
    ========== RESTART: C:\Users\CSD\Desktop\cs\w3q1.py ===
    Enter a number: 4
    4 is in the form of 2^k (for k = 2)
>>>
```

2# Write a function to check whether a prime number is Mersenne prime or not?

```
def mersenne_prime():
    n = int(input("Enter a prime number: "))
    k = 1
    while (2**k - 1) <= n:
        if 2**k - 1 == n:
            print(f"{n} is a Mersenne prime")
            break
        k += 1
    else:
        print("Not Mersenne prime")

mersenne_prime()
```

```
    ================== RESTART: C:
    Enter a prime number: 5
    Not Mersenne prime
>>>
    ================== RESTART: C:
    Enter a prime number: 7
    7 is a Mersenne prime
>>>
```

3# Write a program to evaluate Euler Totients (Totatives) of a given integer

```
import math
def totatives():
    n = int(input("Enter a number: "))
    count = 0

    for i in range(1, n ):
        if math.gcd(i, n) == 1:
            count += 1
            print(i)

    print(f"Euler Totients of {n} is {count}")

totatives()
```

```
Enter a number: 12
1
5
7
11
Euler Totients of 12 is 4
```

4# Implement a function that takes two arguments (r, n) and computer order of r under modulo(n) operation.

```
import math
def modulus():
    r = int(input("r: "))
    n = int(input("n: "))

    if math.gcd(r, n) != 1:
        print("Order not defined")
    else:
        k = 1
        val = r % n
        while val != 1:
            val = (val * r) % n
            k += 1
    #else:
        #k = 1
        #while pow(r, k, n) != 1:
        #    k += 1
        print(f"Order of {r} modulo({n}) is {k}")

modulus()
```

```
r: 2
n: 7
Order of 2 modulo(7) is 3
```